V.Poojya Lakshmi

192324273

1)

Aim: To develop a Python application using Regular Expressions (`re` module) to validate an email address, password, and mobile number according to predefined security rules.

Program:

```
import re

email = input("Enter Email Address: ")

password = input("Enter Password: ")

mobile = input("Enter Mobile Number: ")

email_pattern = r'^[A-Za-z][A-Za-z0-9._]*@[A-Za-z]+.(com|org|edu|net|in)$'

if re.match(email_pattern, email):

print("Valid Email")

else: print("Invalid Email")

password_pattern = r'^(?=.[A-Z])(?=.[a-z])(?=.\d)(?=.[@#$%&!])[A-Za-z\d@#$%&!]{8,}$'

if re.match(password_pattern, password):

 print("Strong Password")

else: print("Weak Password")

mobile_pattern = r'^[6-9]\d{9}$'
```

```
if re.match(mobile_pattern, mobile

print("Valid Mobile Number")

 else: print("Invalid Mobile Number")
```

```
Valid Email
Strong Password
Valid Mobile Number
```

2)

Aim: To write a Python program to simulate a Deterministic Finite Automaton (DFA) that accepts or rejects input strings based on state transitions. The DFA recognizes strings ending with `"ab"`.

Program:

```
# DFA for strings ending with "ab"

transition = {
   'q0': {'a': 'q1', 'b': 'q0'},
   'q1': {'a': 'q1', 'b': 'q2'},
   'q2': {'a': 'q1', 'b': 'q0'}
}

initial_state = 'q0'
final_states = {'q2'}

string = input("Enter input string: ")

current_state = initial_state
path = [current_state]
```

```
valid = True

for symbol in string:
    if symbol not in {'a', 'b'}:
        valid = False
        break

    current_state = transition[current_state][symbol]
    path.append(current_state)

print("Transition Path:")
print(" → ".join(path))

if valid and current_state in final_states:
    print("Accepted")
else:
    print("Rejected")
```

```
if valid and current_state in final_states:
    print("Accepted")
else:
    print("Rejected")
```

```
Enter input string: aabab
Transition Path:
q0 → q1 → q1 → q2 → q1 → q2
Accepted
```

3)

Aim: To develop a Python-based text search engine using Regular Expressions (`re` module) to search for dates, phone numbers, hashtags, mentions, prefixes, and suffixes in a given text.

Program:

```
import re

text = """Meeting on 12/09/2026
Call 9876543210
#NLP
@OpenAI
natural language processing"""

print("1. Search Date")
print("2. Search Phone Number")
print("3. Search Hashtag")
print("4. Search Mention")
print("5. Search Prefix")
print("6. Search Suffix")
print("7. Search Word")

choice = input("Enter your choice: ")

if choice == '1':
    pattern = r'\b\d{2}/\d{2}/\d{4}\b'
elif choice == '2':
    pattern = r'\b[6-9]\d{9}\b'
elif choice == '3':
    pattern = r'#\w+'
elif choice == '4':
    pattern = r'@\w+'
elif choice == '5':
    prefix = input("Enter prefix: ")
    pattern = r'\b' + re.escape(prefix) + r'\w*'
elif choice == '6':
    suffix = input("Enter suffix: ")
    pattern = r'\b\w*' + re.escape(suffix) + r'\b'
elif choice == '7':
    word = input("Enter word to search: ")
    pattern = r'\b' + re.escape(word) + r'\b'
else:
    print("Invalid choice")
```

```
    pattern = None

if pattern:
    result = re.findall(pattern, text, re.IGNORECASE)

    if result:
        print("Matching patterns:")
        for item in result:
            print(item)
    else:
        print("No matching patterns found.")
```

```
        print("Matching patterns:")
        for item in result:
            print(item)
    else:
        print("No matching patterns found.")
```

```
1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Search Word
Enter your choice: 1
Matching patterns:
12/09/2026
```